

# ATM Exception Handling

Difficulty: Easy

Expect Time: 15 min

Author: Homework

A function that returns a special error code is usually accomplished by throwing an exception.

The following class maintains an account balance:

```
class Account
{
private:
    double balance;
public:
    Account () { balance = 0; }
    Account (double initialDeposit) { balance = initialDeposit; }
    double getBalance () { return balance; }

    // returns new balance or -1 if error
    double deposit (double amount)
    {
        if (amount > 0) balance += amount;
        else return -1;
        return balance;
    }

    // return new balance or -1 if invalid amount
    double withdraw (double amount)
    {
        if ((amount > balance) || (amount < 0)) return -1;
        else balance -= amount;
        return balance;
    }
};
```

Rewrite the class so that it throws appropriate exceptions instead of returning **-1** as an error code.

## Exception Handling Operations

You should implement the following classes to handle exceptions:

- **NegativeDeposit**: Handling errors generated by the function `deposit(double)` when the deposit amount is negative.
- **InvalidAmount**: Handling errors when the input amount is not a valid non-negative integer (e.g., decimals like `30.5`).
- **InsufficientFunds**: Handling errors generated by the function `withdraw(double)` when the withdrawal amount exceeds the current balance.
- **NegativeWithdrawal**: Handling errors generated by the function `withdraw(double)` when the withdrawal amount is negative.

## Expected Output When Catching Exceptions

- **NegativeDeposit** : You may only deposit a positive amount.
- **InvalidAmount** : Amount must be a non-negative integer.
- **InsufficientFunds** : Not enough money to withdraw that amount.
- **NegativeWithdrawal** : Withdrawal amount cannot be negative.

## Input

1. Complete the required class implementation. The following files already contain some implementations:
  - `Account.h`
2. The Online Judge system will replace the following files. **DON'T** write implementation in those file:
  - `entrypoint.cpp`
  - `test.h`
  - `case.h`
3. The test cases for student available in `case*.h` .

## Output

Online Judge will run the test:

1. When any test failed, Online Judge will try to list error messages.
2. The Online Judge will capture your output. Please refer to the `expected` variable in `case*.h`.